

LifeThread

Build Specification

For Kiro -- MVP v1.0

A personal life companion. Not a productivity tool.

1. Product Overview

LifeThread is a single-page web app where every project, idea, task, habit, and responsibility in your life becomes a Thread with a chronological timeline of Entries.

Status is never set manually -- it is inferred from the latest Entry type. The Daily Feed is the primary entry point: a reverse-chronological stream of everything you touched today.

1.1 One-Sentence Pitch

A living record of everything that matters -- not just what you need to do, but what you have thought, built, learned, postponed, completed, and forgotten.

1.2 Guiding Principles

- Capture in under 5 seconds

- Never lose context -- everything has history

- Status is inferred, not maintained

- Calm over cluttered -- no red badges, no urgency theater

- Your data belongs to you -- export to Markdown/JSON

2. Data Model

2.1 Thread

```
id: UUID (primary key)
title: string (required, max 200 chars)
folder: enum ['Life', 'Doing', 'Ideas', 'Archive']
subfolder: string (optional, e.g. 'health', 'active projects')
tags: string[] (e.g. ['work', 'finance'])
created_at: ISO timestamp
updated_at: ISO timestamp (touches on every new entry)
archived_at: ISO timestamp | null
abandoned_at: ISO timestamp | null
```

2.2 Entry

```
id: UUID (primary key)
thread_id: UUID (foreign key -> Thread)
type: enum ['log', 'note', 'blocker', 'waiting', 'decision', 'milestone', 'completed', 'attachment']
body: string (required, max 2000 chars)
attachment_url: string | null
created_at: ISO timestamp
```

2.3 Tag

```
id: UUID (primary key)
name: string (unique, lowercase, e.g. 'work', 'health')
color: string (hex, auto-assigned from palette)
created_at: ISO timestamp
```

2.4 Status Inference Rules

Status is computed on read, never stored. Rules evaluated in order:

1. If thread.archived_at exists -> ARCHIVED
2. If thread.abandoned_at exists -> ABANDONED
3. If latest_entry.type == "completed" -> DONE
4. If latest_entry.type == "blocker" -> BLOCKED
5. If latest_entry.type == "waiting" -> WAITING
6. If latest_entry.type in ["log", "note", "decision", "milestone", "attachment"] -> ACTIVE
7. If no entries -> INBOX

If latest entry is older than 14 days and status is ACTIVE, show a "DORMANT" badge in UI but keep ACTIVE as the canonical status.

3. Screens & Components

3.1 Layout Shell

Persistent left sidebar (240px) + main content area. Mobile: sidebar becomes bottom nav.

Sidebar Navigation

Daily Feed -- Primary entry point. Reverse-chronological entries across all threads.

Open Threads -- All threads with status INBOX, ACTIVE, WAITING, BLOCKED. Grouped by folder.

Ideas Park -- Folder == "Ideas". Low-pressure space for not-yet-tasks.

Archive -- Folder == "Archive" + threads with status DONE. Browsable by month.

Tags -- All tags. Clicking filters Open Threads by tag.

Search -- Full-text search across thread titles and entry bodies.

3.2 Daily Feed (Primary Screen)

This is the app's home screen. It must feel alive, not like a database.

Layout

Top: Quick-capture input box (sticky, always visible)

Below: Reverse-chronological list of entries from today, grouped by thread

Each entry card shows: thread title (clickable), entry type badge, body text, timestamp

Thread title is a link that opens the Thread Detail view

Quick Capture Input

A single text input at the top of the feed. Behavior:

User types and hits Enter -> app shows a dropdown: "Add to existing thread" or "Create new thread"

If user selects existing thread -> creates a Log entry on that thread

If user selects "New thread" -> opens a modal: title, folder, tags. Then creates thread + Log entry

Typing "#tagname" in the input auto-suggests tags

Feed Entry Cards

```
[Log]  Start YouTube channel
      "Bought Rode microphone -- $99 on Amazon"
      2 hours ago

[Blocker]  Fix router issue
          "ISP says outage until Friday 6pm"
          5 hours ago
```

3.3 Thread Detail View

Opened by clicking a thread title anywhere in the app.

Header Section

Thread title (editable inline)

Current status badge (inferred, read-only)

Folder path (e.g. "Doing > active projects")

Tag pills (click to remove, + button to add)

Action buttons: Add Entry, Archive, Abandon, Edit Title

Timeline

Vertical timeline of all entries, newest at top

Each entry: type badge + body + timestamp + optional attachment preview

Entry types render with distinct colors:

Log	-- neutral gray
Note	-- soft blue
Blocker	-- soft red (not alarming)
Waiting	-- soft amber
Decision	-- soft green
Milestone	-- soft purple
Completed	-- soft teal

Inline "Add Entry" form at the top of the timeline: select type (default Log), text area, submit

3.4 Open Threads View

A browseable list of all non-archived, non-abandoned threads.

Grouping

Grouped by folder (Life, Doing, Ideas)

Within folder, grouped by subfolder if present

Within group, sorted by updated_at desc (most recently touched first)

Thread List Item

```
[ACTIVE] Meal prep plan
         #health | 3 entries | Updated 2h ago

[WAITING] Fix router issue
         #home | 5 entries | Updated 5h ago

[BLOCKED] Start YouTube channel
         #content | 2 entries | Updated 1d ago
```

Filtering

Top bar: filter by status (All, Active, Waiting, Blocked, Inbox)

Tag filter pills below status filter

Search input filters thread titles in real-time

3.5 Ideas Park

Functionally identical to Open Threads but pre-filtered to folder == "Ideas".

Visual difference: softer background tint, encouraging exploration over execution.

3.6 Archive View

All threads with status DONE, ARCHIVED, or ABANDONED.

Grouped by month of last update (e.g. "July 2026", "June 2026")

Each item shows: status badge, title, entry count, final entry preview

Click to open Thread Detail (read-only mode, or allow adding new entries to revive)

3.7 Search

Full-text search across:

Thread titles

Entry bodies

Tag names

Results grouped by thread, showing matching entry snippets with highlighted terms.

4. Key User Flows

4.1 Create a New Thread

1. User types in Quick Capture: "App idea for cats"
2. Hits Enter -> selects "Create new thread"
3. Modal opens:
 - Title: "App idea for cats" (pre-filled from input)
 - Folder: dropdown (default "Ideas")
 - Subfolder: text input (optional)
 - Tags: multi-select (typeahead)
4. User hits "Create"
5. Thread created + first Entry (type: log, body: "App idea for cats") created
6. User lands on Thread Detail view

4.2 Add an Entry to an Existing Thread

1. User types in Quick Capture: "called plumber, waiting on callback"
2. Hits Enter -> app suggests "Fix router issue" (fuzzy match on recent threads)
3. User selects it
4. Entry created (type: log, body: "called plumber, waiting on callback")
5. Thread status auto-updates to WAITING (because body contains "waiting")
OR user manually selects "waiting" type before submitting

Smart type detection: if entry body contains "waiting" / "blocked" / "done", suggest matching entry type. User can override.

4.3 Archive a Thread

1. User opens Thread Detail
2. Clicks "Archive" button in header
3. `thread.archived_at = now()`
4. Thread disappears from Open Threads, appears in Archive
5. User can unarchive from Archive view

4.4 Revive a Thread

1. User finds an archived thread in Archive view
2. Clicks "Revive" or adds a new entry
3. `thread.archived_at = null`
4. Thread reappears in Open Threads with status ACTIVE

5. Recommended Tech Stack

5.1 Frontend

Framework: React 19 + TypeScript

Styling: Tailwind CSS

State: Zustand (lightweight, no boilerplate)

Routing: React Router (or file-based if using Next.js)

Date handling: date-fns

Icons: Lucide React

5.2 Backend (if building with a backend)

Option A -- Fullstack: Next.js 15 (App Router) + Prisma + SQLite (local-first)

Option B -- API + SPA: Express/Fastify + PostgreSQL + React SPA

Option C -- Local-first (recommended for MVP): React + IndexedDB (via Dexie.js) + export to JSON/Markdown

5.3 Data Persistence Strategy

For MVP, recommend local-first with optional cloud sync later:

All data stored in browser IndexedDB via Dexie.js

Instant load, works offline

Export button generates a .json file (full backup) and .md files (one per thread)

Future: optional Supabase/Firebase sync layer

5.4 Project Structure

```
src/
  components/
    Layout.tsx          -- Sidebar + main area shell
    QuickCapture.tsx     -- Sticky input at top of Daily Feed
    EntryCard.tsx        -- Single entry in feed or timeline
    ThreadListItem.tsx   -- Row in Open Threads / Archive
    ThreadDetail.tsx     -- Full thread view with timeline
    StatusBadge.tsx      -- Colored badge for inferred status
    TagPill.tsx          -- Clickable tag component
    EntryTypeSelector.tsx -- Dropdown for entry type
  stores/
    threadStore.ts       -- Zustand store for threads + entries
  hooks/
    useStatus.ts         -- Compute status from thread + entries
    useDailyFeed.ts      -- Aggregate entries for feed
    useSearch.ts         -- Full-text search
  types/
    index.ts            -- Thread, Entry, Tag types
  utils/
    statusInference.ts   -- Status rules engine
    export.ts            -- JSON + Markdown export
    dateFormat.ts        -- Relative time formatting
```

6. Build Phases

Phase 1: Core Loop (Days 1-3)

- Data model + Dexie.js schema
- Thread CRUD (create, list, detail view)
- Entry CRUD inside thread detail
- Status inference engine
- Basic sidebar navigation

Phase 2: Daily Feed (Days 4-5)

- Quick Capture input with thread suggestion
- Reverse-chronological feed of all entries
- Entry cards with type badges and timestamps
- Click thread title -> navigate to detail

Phase 3: Organization (Days 6-7)

- Folder / subfolder structure
- Open Threads grouped view
- Tag system (create, assign, filter)
- Archive view with monthly grouping

Phase 4: Polish (Days 8-10)

- Full-text search
- Export to JSON + Markdown
- Smart type detection in Quick Capture
- Dormant thread warnings (14-day inactive)
- Responsive mobile layout
- Onboarding empty states

7. Acceptance Criteria

Must Have (P0)

- [] User can create a thread with title, folder, and tags
- [] User can add entries to a thread with a type and body
- [] Status is correctly inferred from latest entry type
- [] Daily Feed shows all entries reverse-chronological
- [] Quick Capture creates entries with < 5 second friction
- [] Open Threads groups by folder and sorts by recency
- [] Archive shows completed/abandoned threads by month
- [] Search finds threads and entries by text
- [] Data persists across browser sessions (IndexedDB)
- [] Export produces valid JSON and Markdown files

Nice to Have (P1)

- ☐ Smart type detection from entry text ("waiting" -> Waiting)
- ☐ Dormant badge on threads inactive > 14 days
- ☐ Entry attachments (image paste / file drop)
- ☐ Keyboard shortcuts (Cmd+K quick capture, / for commands)
- ☐ Dark mode

Future (P2+)

- ☐ Habits module with streak tracking
- ☐ Thread linking (parent/child, related)
- ☐ Weekly auto-review generation
- ☐ Cloud sync
- ☐ Mobile app (React Native or PWA)

8. Design Notes

8.1 Visual Language

Typography: Inter or system-ui stack

Spacing: generous whitespace, 16px base grid

Colors: muted, calm palette. No pure reds or greens. Soft pastels for entry types.

Animations: subtle fade-in for new entries, 150ms transitions

8.2 Color Palette (Entry Types)

Log	-- #6B7280 (gray-500)
Note	-- #3B82F6 (blue-500, muted)
Blocker	-- #EF4444 (red-500, muted to rose-400)
Waiting	-- #F59E0B (amber-500, muted)
Decision	-- #10B981 (emerald-500, muted)
Milestone	-- #8B5CF6 (violet-500, muted)
Completed	-- #06B6D4 (cyan-500, muted)

8.3 Empty States

Daily Feed (no entries yet): "Your day is a blank page. What happened?" + large capture input

Open Threads (none): "Nothing on your plate. Check Ideas Park or start something new."

Archive (none): "You have not finished anything yet. That is okay."

8.4 Mobile Behavior

Bottom nav replaces sidebar: Feed | Open | Ideas | Archive | Search

Quick Capture becomes a floating action button (+) that expands to input

Thread Detail is a full-screen modal with swipe-down to close